

Group 4: John Christian Ballesteros, Isaiah S. Collado, Uno Rayven Timothy Kabling, Jeremiah Daniel C. Padilla, Carlos Gabriel D. Roman

Repository link:

<https://github.com/IsaiahCollado/Programming-for-Data-Science-Final-Project>

Commit identifier: 9cfa90b

Title: The Analysis of Metro Manila Vehicular Crash Patterns: Area and Demographic Evaluation of MMDA Incident Data

Intended Audience: Metropolitan Manila Development Authority (MMDA)

Specific Problem: Total Traffic Disruptions and Collision Hazards Across Metro Manila LGUs

Analytical Questions:

1. How do temporal peak windows (rush hour vs. off-peak) and collision densities across Metro Manila cities shift between standard workdays (Monday–Friday) and weekends?
2. To what degree do differences in urban density explain variations in localized collision rates across Metro Manila LGUs, and which cities deviate most from this trend as high-risk anomalies?"
3. What proportion of total recorded roadway disruptions in Metro Manila is attributable to vehicular stalls and mechanical failures compared to active collisions, and which specific vehicle categories are most susceptible to operational breakdowns?

Dataset 1 (Traffic Data):

File Name & Provider: data_mmda_traffic_spatial.csv (Compiled by Kaggle user esparko from Metropolitan Manila Development Authority / MMDA official Twitter/X traffic incident alerts).

URL: <https://www.kaggle.com/datasets/esparko/mmda-traffic-incident-data>

Coverage: 2018-2020

Conditions: Released under the Open Data Commons Public Domain Dedication and License (PDDL) / Open Database License (ODbL) for academic, research, and non-commercial analytical use.

Relevant Fields: Date, Time, City, Location, Direction, Type, Involved.

Integration Key: City.

Dataset 2 (LGU Data):

File/Provider: metro_manila_local_government.csv (Extracted from the administrative table on Wikipedia, citing Philippine Statistics Authority (PSA) Census figures and official LGU administrative records).

URL: https://en.wikipedia.org/wiki/Metro_Manila

Coverage: 2024 Population Data

Conditions: Published under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0) and GNU Free Documentation License (GFDL).

Relevant Fields: City or municipality, Population (2024), Area (km²), Density (per km²).

Integration Key: City or municipality.

Inspection: Traffic shape is (17312, 13); LGU is (18, 8). Types are object, float64, int64. No duplicates. Missing values found in Direction (857) and Lanes_Blocked (687).

Variables:

Q1: Date, Time, City

Q2: City, City or municipality, Density (per km²)

Q3: Type and Involved

Program Components: (P: Primary Programmer, R: Reviewer)

Acquisition (P: Collado | R: Padilla): Read CSVs using `pandas.read_csv()`.

Validation (P: Roman | R: Collado): Check types and nulls using `df.info()` and `df.isnull()`.

Cleaning (P: Padilla | R: Roman): Remove special characters, convert Time to datetime.

Integration (P: Padilla | R: Kabilig): Merge dataframes on City names.

Analysis (P: Roman | R: Collado): Use `groupby()` and math aggregations.

Visualization (P: Kabilig | R: Ballesteros): Generate charts with `matplotlib/seaborn`.

Testing (P: Ballesteros | R: Kabilig): Verify merge integrity using `assert`.

Execution (P: Padilla | R: Roman): Run Jupyter Notebook cleanly top-to-bottom.

Risk Assessment:

Mismatched Keys: Clean strings using `str.replace()` before merging.

Data Quality (Vehicles): Use `str.contains()` to create boolean flags for messy text.

Missing Info: Impute missing Lanes_Blocked with the median, or drop rows.

Privacy: Drop Tweet column to avoid PII.

Scope: Reduce analysis to the top 5 LGUs if needed.

Schedule: Drop Q3 if falling behind the deadline.

Schedule & Feedback

Week 8: Proposal & Check-in 1

Week 9: Prelims

Week 10: Validation & Cleaning

Week 11: Integration

Week 12: Advanced Analysis

Week 13: Visualization

Week 14: Testing

Week 15: Documentation

Week 16: Review

Week 17: Final Submission

Questions for Instructor:

1. Is `str.contains()` sufficient for categorizing the highly irregular vehicle column?
2. Should we clean LGU special characters in the CSV or strictly via Python?
3. Should we drop or impute rows with missing `Lanes_Blocked` values?